

Enhancing privacy-preserving machine learning with self-learnable activation functions in fully homomorphic encryption

Jichao Xiong^{a,b}, Jiageng Chen^{a,b,*}, Junyu Lin^{a,b}, Dian Jiao^{a,b}, Hui Liu^a

^a School of Computer Science, and Hubei Provincial Key Laboratory of Artificial Intelligence and Smart Learning, Central China Normal University, Wuhan, 430079, Hubei, China

^b School of Central China Normal University Wollongong Joint Institute, Central China Normal University, Wuhan, 430079, Hubei, China

ARTICLE INFO

Keywords:

Fully homomorphic encryption
HE-friendly neural network
Privacy-preserving machine learning

ABSTRACT

In the field of artificial intelligence and data engineering, the effective utilization of data is critical for improving productivity across various sectors. As machine learning increasingly relies on sensitive data, balancing privacy with computational efficiency has become a major challenge. Homomorphic encryption provides a promising solution by enabling computation on encrypted data while preserving privacy in machine learning. However, its integration with neural networks is hindered by high computational demands and limitations in performing complex nonlinear operations. To address these challenges, we propose a novel approach that incorporates a "Self-Learnable Activation Function" (SLAF) and refines the structure of neural network linear layers. These enhancements are designed to accommodate the constraints of homomorphic encryption, allowing for deeper network architectures without significant computational overhead.

Our optimized neural network model, tailored for biometric authentication tasks, outperforms traditional methods that use simple polynomial activation functions. Using the UTKFace dataset, which includes facial features under diverse scenarios, we validated the effectiveness of our solution in real-world applications. Experimental results demonstrate accuracy improvements of 0.88% to 3.15% over traditional models and 4.87% to 9.67% over the CryptoNets model, underscoring the capability of our approach to meet stringent privacy-preserving biometric authentication requirements.

1. Introduction

In the dynamic landscape of artificial intelligence (AI) technology, the strategic use of data significantly boosts productivity across numerous sectors. Nonetheless, a crucial conflict emerges between the imperative of data privacy protection and the necessity for secondary data use in AI applications, such as deep learning. This conflict is primarily due to the differing needs of data owners and users: while data users seek access to private data for analytical and processing purposes, data owners are obligated to maintain data confidentiality. This scenario presents a complex challenge in modern data use: ensuring that private data is both accessible for user operations or cloud-based computational tasks and protected to maintain owner confidentiality.

Homomorphic encryption represents an important privacy-preserving solution by enabling operations like addition and multiplication on encrypted data without decryption. This capability ensures that data remains encrypted throughout processing, effectively safeguarding the owner's sensitive information during both transmission and computation. As such, homomorphic encryption not only meets the

computational demands for handling encrypted data but also supports its application in fields like image analysis using Convolutional Neural Networks (CNN).

At present, homomorphic encryption technology has been widely applied in the field of biometrics to achieve efficient and secure privacy protection. For instance, Morampudi et al. developed a secure and verifiable iris authentication system based on classification using homomorphic encryption [1]. Choi et al. addressed privacy concerns related to storing and processing biometric data on servers by implementing fingerprint authentication on encrypted data through homomorphic encryption [2]. Pradel et al. proposed a homomorphic encryption-based biometric authentication protocol, where biometric samples are collected by local devices, and the remote server performs template matching on encrypted data. These studies demonstrate the significant application prospects and practical value of homomorphic encryption technology in protecting biometric data privacy [3].

Despite the advantages of homomorphic encryption in preserving data privacy, it is not without its limitations, which can restrict its

* Corresponding author.

E-mail address: jiageng.chen@ccnu.edu.cn (J. Chen).

<https://doi.org/10.1016/j.jisa.2024.103887>

practicality [4]. The significant computational overhead and time required for operations under this encryption method lead to efficiency drawbacks compared to processing unencrypted data, particularly in model training and prediction contexts. Furthermore, homomorphic encryption is limited to basic operations such as addition and multiplication, complicating the implementation of deep learning's nonlinear activation functions. Compounding this issue, each operation in homomorphic encryption introduces noise, which can accumulate and degrade result accuracy. Excessive noise might prevent the correct decryption of the ciphertext. Therefore, homomorphic encryption-friendly networks are typically constrained by hierarchical structures and tend to favor low-degree polynomials as activation functions.

1.1. Privacy-preserving machine learning

CryptoNets [5] is a scheme that combines deep learning and homomorphic encryption techniques for neural network computation on encrypted data. This scheme aims to protect user privacy by performing machine learning operations on encrypted data in the cloud without revealing sensitive information. CryptoNets constructs a simple 5-layer network to predict the likelihood value of the MNIST [6] dataset ($28 \times 28 \times 1$) with 99% accuracy despite using low-precision training, approximated activation functions, and shallow network structure.

Privacy-Preserving Machine Learning techniques have advanced with the work of Joon-Woo Lee et al. [7], addressing efficiency issues in traditional Fully Homomorphic Encryption (FHE)-based models like CryptoNet [5], SEALion [8], and CryptoDL [9]. Their method, which incorporates an approximation technique for non-arithmetic activation functions coupled with bootstrapping, allows continuous homomorphic evaluation. The approach is validated on more practical and complex datasets, specifically using the standard ResNet-20 model and the RNS-CKKS FHE scheme on the CIFAR-10 dataset, showcasing both efficiency and the maintenance of explicit model parameters.

CRYPTEN [10] introduces a software framework that extends the usability of popular secure multi-party computation primitives through abstraction techniques common in modern machine learning frameworks, such as tensor computations, automatic differentiation, and modular neural networks. Notably, its GPU capabilities enhance performance in scenarios involving multiple parties, facilitating efficient and private evaluation of advanced machine learning models under a semi-honest threat model.

HybridAlpha [11] presents a novel approach to privacy-preserving federated learning that leverages functional encryption to enhance secure multi-party computation protocols. This method effectively reduces the risk posed by curious aggregators and colluding participants, through the use of differential privacy mechanisms and the creation of secure multi-party computation protocols from multi-input functional encryption schemes. It demonstrates a significant reduction in training time by 68% and data transmission volume by 92%, while still matching the model performance and privacy guarantees of existing cryptographic solutions.

In contrast, several other techniques utilize homomorphic encryption [12,13], federated learning [14], and secure multi-party computation [15] to preserve privacy in machine learning, while hybrid schemes [16–18] combine multiple techniques to enhance privacy preservation. This paper primarily focuses on optimizing the use of homomorphic encryption within neural networks to further the capabilities of privacy-preserving machine learning.

1.2. Activation function with homomorphic encryption

Current homomorphic encryption frameworks such as BGV [19], FV [20], TFHE [21], and CKKS [22] offer diverse approaches to handling nonlinear activation functions, which are crucial for deep learning applications. Among these, TFHE [21] is particularly notable for its capability to implement nonlinear activation functions directly on ciphertexts using look-up table evaluations. In contrast, the CKKS scheme [22]

Table 1

A comprehensive overview of activation function implementations in common homomorphic encryption schemes.

HE network framework	Activation function realization
CryptoNets [5] SEALion [8] HCNN [23] BAYHENN [24]	Square activation function
Faster CryptoNets [25] CryptoDL [9] JUNGHYUN LEE et al. [26] TensorHE [27] CHET [28]	Polynomial function approximation
PEGASUS [29] CHIMERA [30]	Evaluate look-up tables on FHEW ciphertexts

faces significant challenges in this area, prompting the need for innovative solutions for practical deployment. Consequently, alternative strategies have been developed. For instance, simpler functions like the square activation function is used in models such as CryptoNets [5], SEALion [8], HCNN [23], and BAYHENN [24], serving as effective substitutes for traditional nonlinear activation functions.

To enhance the approximation of nonlinear activation functions, specifically the ReLU function, various encryption schemes have adopted polynomial approximations. The Faster CryptoNets model [25] applies a quantized polynomial approximation, specifically $2^{-3}x^2 + 2^{-1}x + 2^{-2}$, for the ReLU function. CryptoDL [9] employs a cubic polynomial for similar purposes, while JUNGHYUN LEE et al. [26] utilize minimax approximate polynomials of low degrees. TensorHE [27] adopts a comparable approach to CryptoDL [9] in implementing activation functions. In a unique strategy, the CHET scheme [28] models the activation function as a quadratic polynomial, $ax^2 + bx$, with coefficients a and b adjustable during training. Beyond these polynomial methods, PEGASUS [29] and CHIMERA [30] employ an innovative hybrid technique that combines ciphertexts from both FHEW and RLWE frameworks. This integration facilitates efficient arithmetic operations under RLWE, while FHEW handles the activation functions, exemplifying the varied strategies within homomorphic encryption to support complex activation functions, as summarized in Table 1.

1.3. Our contribution

Optimization of Neural Network Architectures with Homomorphic Encryption: The integration of homomorphic encryption into neural network architectures often leads to significantly increased computational demands, particularly as the network deepens. To mitigate this, we optimized the structure of the network's linear layers, which effectively controlled the escalation in computational time and facilitated greater network depth. Moreover, we introduced a novel activation function, termed the Self-Learnable Activation Functions (SLAF), specifically designed to be compatible with homomorphic encryption. This innovation aims to address the precision loss commonly associated with homomorphic encryption-friendly networks, which typically employ low-degree polynomial functions.

Development of a Privacy-Preserving Machine Learning Framework Using CNN: We have developed an integrated framework that combines homomorphic encryption with Convolutional Neural Networks (CNN) to support privacy-preserving machine learning. To accommodate various operational needs, we designed two distinct neural network models: one optimized for rapid response and the other for high precision. This framework was extensively tested in the realm of biometric identification, utilizing the UTKFace database [31] for a series of classification tasks, including age, gender, and ethnicity. We benchmarked the performance of our models against existing solutions to assess their effectiveness and practical utility.

Performance and Efficacy of the Homomorphic Encryption-Compatible Activation Function: The performance analysis reveals that our homomorphic encryption-compatible activation function, which was specifically trained across various datasets and network configurations, significantly enhances model accuracy. Compared to models utilizing square activation function, our model with the SLAF activation function showed improvements ranging from 0.88% to 3.15% in accuracy for encrypted data processing. While there was a minor accuracy decrease of 0.45% to 0.84% in gender and age classifications compared to models using traditional ReLU functions, there was a noteworthy accuracy gain of 0.96% in ethnicity classification tasks. Remarkably, our high-precision model surpassed the CryptoNets model's accuracy by 4.87% to 9.67%, and our rapid-response model demonstrated an accuracy improvement of 1.17% to 5.14%. Performance evaluations on an Apple M1 Pro CPU (8 cores) with 16 GB of memory underscored the low computational overhead of the rapid-response model, which was just 0.778 s, while the high-precision model required approximately 1 min. These research results confirm the excellent performance characteristics of our proposed machine learning method, demonstrating its effectiveness and privacy protection capabilities in biometric authentication.

2. Preliminaries

2.1. Fully homomorphic encryption

The concept of Fully Homomorphic Encryption (FHE) was originally known as privacy homomorphism and was introduced by Rivest et al. in 1978 [32]. FHE is a special type of encryption that allows computations to be performed on encrypted data without the need for decryption to obtain the result. This means that operations such as addition, multiplication, etc. can be performed even in the encrypted state and the result obtained can be decrypted to match the result obtained by performing the same operation on unencrypted data. Homomorphic encryption allows data operations to be performed in the encrypted domain, therefore, FHE has great potential and promise in privacy protection. It provides a strong privacy protection mechanism in areas such as machine learning, cloud computing, data processing, and shared data.

The actual FHE scheme was first proposed and implemented by Craig Gentry only in 2009 [33]. However, the initial scheme was very inefficient, with extremely high computational costs, and not very suitable for practical applications. After this, FHE attracted wider attention and became one of the research hotspots in the field of cryptography and security. Since then, researchers have continuously improved and optimized the FHE schemes, and various improved versions and new concepts have been proposed, among which the most representative ones are BGV [19], FV [20], TFHE [21] and CKKS [22].

In the FHE scheme, the noise grows as the computation increases, which may ultimately lead to a failure to restore the result correctly when decrypting. Bootstrapping provides a way to allow re-encryption of encrypted data while in an encrypted state, while removing or reducing the noise, thus allowing the data to remain computable. Although bootstrapping is an important technological breakthrough, it is computationally expensive and usually requires more computational resources and time. Therefore, a more efficient variant of the FHE scheme, Levelled-FHE [19], which can be configured to support computations with a certain multiplicative depth without bootstrapping, is often adopted in practice.

In this study, we used CKKS levelled-FHE implemented in TenSEAL [34], a python wrapper library for Microsoft SEAL [35], in all our experiments.

2.2. CKKS FHE scheme

The CKKS homomorphic encryption scheme, based on the Ring Learning With Errors (RLWE) problem, is an approximate arithmetic

homomorphic encryption method [22]. Unlike BGV [19] and FV [20], whose plaintext spaces consist of integer polynomials, CKKS employs a complex space as its message space. Consequently, it supports approximate addition and multiplication operations on encrypted data and introduces a novel scaling procedure to manage the scale of plaintexts. Neural networks typically involve extensive floating-point calculations. Therefore, in this study, we opt for the CKKS scheme, which supports efficient floating-point computations and results in smaller ciphertext sizes. The CKKS homomorphic encryption algorithm consists primarily of several components: key generation, encryption, decryption, homomorphic addition, homomorphic multiplication, and rescaling.

- $\text{KeyGen}(1^\lambda) \rightarrow \{\text{SK}, \text{PK}, \text{EK}\}$: The process of key generation (KeyGen) involves taking the encryption parameters as input and producing a set of keys: the secret key (SK), mainly employed for decryption; the public key (PK), utilized for encryption; and the evaluation key (EK), is used to evaluate homomorphic operations on ciphertexts.
- $\text{Encrypt}(\text{PK}, M) \rightarrow C$: The Encrypt operation takes as input PK and a plaintext message M from the plaintext space $\mathcal{P}$, and returns a valid ciphertext C from the ciphertext space $\mathcal{C}$.
- $\text{Decrypt}(\text{SK}, C) \rightarrow M$: The Decrypt operation receives as input the SK and a valid ciphertext C from the ciphertext space $\mathcal{C}$, which decrypts the message M in the plaintext space $\mathcal{P}$, and outputs the original message M.
- $\text{HomoAdd}(\text{EK}, C^{(1)}, C^{(2)}) \rightarrow C^{(3)}$: The HOMOAdd operation accepts the EK, and two valid ciphertexts, $C^{(1)}$ encrypting $M^{(1)}$ and $C^{(2)}$ encrypting $M^{(2)}$. It returns a valid ciphertext $C^{(3)}$ encrypting the result $(M^{(1)} + M^{(2)})$.
- $\text{HomoAddConst}(\text{EK}, C^{(1)}, M^{(2)}) \rightarrow C^{(3)}$: The HOMOAddConst operation accepts the EK, $C^{(1)}$ encrypting $M^{(1)}$ and $M^{(2)}$, and returns a valid ciphertext $C^{(3)}$ encrypting the result $(M^{(1)} + M^{(2)})$.
- $\text{HomoMult}(\text{EK}, C^{(1)}, C^{(2)}) \rightarrow C^{(3)}$: The HOMOmult operation accepts the EK, and two valid ciphertexts, $C^{(1)}$ encrypting $M^{(1)}$ and $C^{(2)}$ encrypting $M^{(2)}$. It returns a valid ciphertext $C^{(3)}$ encrypting the result $(M^{(1)} \times M^{(2)})$.
- $\text{HomoMultConst}(\text{EK}, C^{(1)}, M^{(2)}) \rightarrow C^{(3)}$: The HOMOmultConst operation accepts the EK, $C^{(1)}$ encrypting $M^{(1)}$ and $M^{(2)}$, and returns a valid ciphertext $C^{(3)}$ encrypting the result $(M^{(1)} \times M^{(2)})$.
- $\text{Rescale}(C, \Delta) \rightarrow \hat{C}$: The Rescale operation involves a valid ciphertext M and scaling it down by Δ to produce an equivalent ciphertext $\hat{C}$. This resultant ciphertext $\hat{C}$ encrypts the same plaintext but with a reduced scale factor and diminished noise level.

3. Design efficient privacy-preserving neural network models with homomorphic encryption

Utilizing Fully Homomorphic Encryption (FHE) in Convolutional Neural Networks (CNN) significantly increases both resource demand and computational time. FHE is a potent encryption method that permits computations on encrypted data without necessitating decryption. However, its application in deep learning models like CNN introduces increased complexity and higher computational costs. In conventional CNN, convolutional operations serve as fundamental components; however, these operations become considerably more resource-intensive under FHE. This is primarily due to the high resource demands of performing basic operations such as multiplication and addition on encrypted data. Specifically, multiplication operations under FHE consume more resources and time compared to addition operations. To mitigate the impact of FHE on CNN efficiency, we have optimized the convolutional and fully connected layer operations. By reducing the number of multiplications required, we have effectively decreased the overall runtime.

The constraints imposed by the CKKS scheme [22], which facilitates only homomorphic addition and multiplication, pose challenges in directly implementing traditional nonlinear activation functions. To

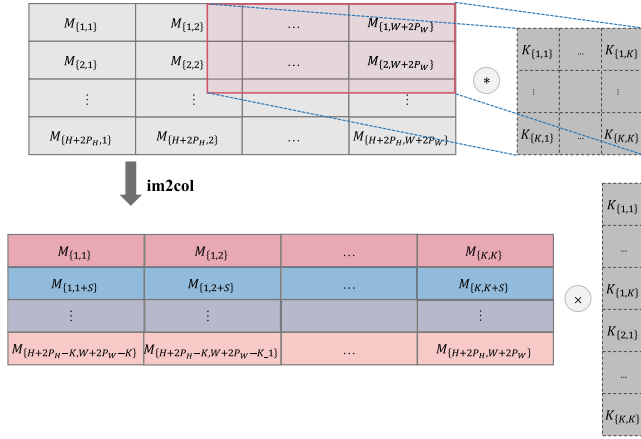


Fig. 1. Conversion of convolution operation to matrix multiplication in homomorphic encryption.

overcome these difficulties and construct a homomorphic encryption-friendly, privacy-preserving CNN model, we have incorporated an activation function with polynomial characteristics. This modification aims to address the accuracy degradation typically associated with the use of low-order polynomials in networks designed to be friendly to homomorphic encryption.

In this paper, we propose a straightforward method that combines the CKKS homomorphic encryption scheme with Convolutional Neural Networks to achieve efficient and privacy-preserving machine learning.

3.1. Construction of homomorphic encryption-friendly convolutional layers

Incorporating homomorphic encryption into neural network architectures significantly escalates computational overhead, particularly for homomorphic multiplication operations, which are more computationally intensive compared to other operations in homomorphic encryption. In the context of a convolutional layer within a homomorphic encryption framework, the computational demands can be exemplified when processing an input image of dimensions W (width) and H (height), using a kernel size of K , a stride of S , and padding of P_h and P_w in the height and width directions respectively. Specifically, a single convolutional layer necessitates a total of $[(H - K + P_h)/S + 1] \times [(W - K + P_w)/S + 1]$ multiplication operations. Minimizing these multiplication operations is crucial for constructing efficient homomorphic encryption-friendly convolutional layers.

Employing the im2col function [36] presents a practical solution for reducing the number of multiplicative operations in homomorphic encryption-based convolutional layers. The im2col function transforms the input data into a matrix format, effectively reducing the multiplicative demands within the convolutional operations to a single operation. Fig. 1 illustrates this transformation, where convolution operations are converted into matrix multiplication using the im2col technique. The resulting matrix from the im2col transformation has a width corresponding to the convolution kernel size, denoted as K^2 , and a height equivalent to the number of times the convolution kernel slides over the input data, represented as $[(H - K + P_h)/S + 1] \times [(W - K + P_w)/S + 1]$. The encoded matrix and the convolution kernel are then sequentially unfolded, resulting in flattened matrices $Enc(M')$ and K' , respectively. The convolution operation is then performed using a homomorphic multiplication, with the computation expressed as:

$$\text{HomoMultConst}(\text{EK}, \text{Enc}(M'), K') \rightarrow \text{Result}_{\text{Conv}}$$

3.2. Construction of homomorphic encryption-friendly linear layers

The operation of a linear layer is equivalent to performing vector-matrix multiplication, followed by the addition of a bias term. In

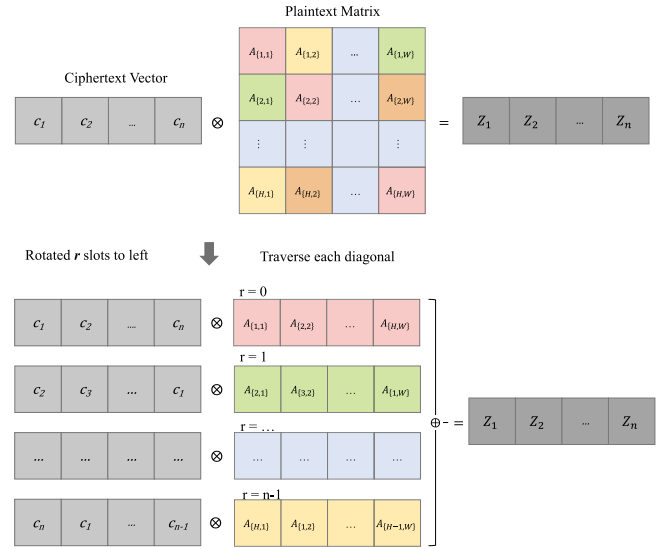


Fig. 2. Linear layer operations in homomorphic encryption schemes.

homomorphic encryption-based neural network models, the matrix multiplication of encrypted data with weight matrices entails significant computational time. In our approach, both the matrices and bias terms utilize weights obtained from plaintext-trained models and remain unencrypted throughout the process. The vector-matrix multiplication operation, based on the Halevi and Shoup diagonal method [37], involves accumulating multiple ciphertext-plaintext multiplications. During computation, each diagonal of the plaintext matrix is traversed, and then multiplied with ciphertext rotated left by r slots, where r represents the index of the diagonal (0-indexed). Specifically, for a Ciphertext Vector C of length n and a weight matrix A with width W and height H , the operation of the linear layer involves rotating C leftward by r slots to obtain C'_r , multiplying it element-wise with each diagonal A'_r obtained by traversing A , and accumulating the results. The operational process is illustrated in Fig. 2.

In the linear layer of homomorphic encryption, the corresponding C'_r is multiplied with A'_r to obtain Z'_r , and then each Z'_r is summed to obtain the final result of the linear layer computation:

$$\sum_{i=1}^r [\text{HomoMultConst}(\text{EK}, \text{Enc}(C'_i), A'_i)] \rightarrow Z'$$

3.3. Construction of homomorphic encryption-friendly activation function

In the integration of homomorphic encryption into neural networks, it is crucial to adapt activation functions that are compatible with homomorphic encryption's operational constraints. Homomorphic encryption schemes typically support only addition and multiplication operations, making the direct implementation of traditional nonlinear activation functions challenging. Consequently, networks that are compatible with homomorphic encryption often employ low-degree polynomial functions, such as square activation function. However, this adaptation can lead to reduced accuracy in model classification and prediction tasks.

In our approach, we leverage the polynomial nature of The Self-Learnable Activation Function (SLAF), which simplifies its implementation within homomorphic encryption schemes, making it particularly suitable for privacy-preserving deep learning. This is crucial in addressing the challenges posed by implementing non-linear activation functions and the accuracy degradation that arises from using square activation functions in homomorphic encryption.

The SLAF was introduced to enhance the understanding and performance of activation functions within neural networks. Traditionally, activation functions like ReLU, Sigmoid, and Tanh are manually designed and often perform inconsistently across different tasks and datasets, limiting their general applicability. SLAF was developed with the goal of learning a generalized activation function that could automatically adapt to specific tasks and datasets during training [38]. However, empirical results have shown that using SLAF alone yields accuracy levels close to those achieved with the ReLU activation function [39].

SLAF's polynomial properties facilitate its implementation in encrypted environments, making it a versatile choice. Unlike other homomorphic encryption approaches that rely on polynomial approximations of functions like ReLU, SLAF can be specifically adjusted and learned for different network models and datasets. This adaptability allows SLAF to effectively meet the demands of various complex scenarios and enhance model accuracy. The general formula for SLAF is as follows:

$$a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_nx^n$$

where $a_0, a_1, a_2, \dots, a_n$ are parameters learned during the training process. Through this form, SLAF not only achieves nonlinear transformations but also provides tailored performance enhancements based on the specific requirements of the applied network and data.

SLAF operates by representing the activation function as a weighted sum of a predefined set of basis functions, which involves four key processes: basis function selection, polynomial representation, normalization, and parameter updating. First, during the basis function selection phase, SLAF utilizes a predefined set of basis functions, such as Taylor polynomial basis and Chebyshev basis, as the foundational elements of the activation function. Next, SLAF expresses the activation function as a weighted sum of these basis functions, with the coefficients for each basis function being adjusted and learned during the training process. Normalization is then applied to mitigate issues like gradient explosion or vanishing gradients, ensuring stable training. The input x is normalized using the following formula:

$$\hat{x}_i = \frac{x_i - \text{mean}(x_i)}{\sqrt{\text{var}(x_i) + \epsilon}}$$

The normalized basis functions are then used in the final SLAF computation, ensuring stability and effectiveness during the training process. Finally, during the training process, the coefficients of the basis functions are iteratively updated using the backpropagation algorithm, allowing SLAF to gradually converge toward an optimal activation function.

Implementing SLAF within a homomorphic encryption scheme requires integrating polynomial construction techniques with the unique characteristics of homomorphic operations. The specific steps involved are as follows:

Polynomial Representation: SLAF expresses the activation function in the form of a polynomial, as shown below:

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_nx^n$$

In a homomorphic encryption environment, the input data x is encrypted, and the polynomial must be evaluated in the encrypted space. Since each term of the polynomial involves exponentiation and weighted summation, these operations are performed within the homomorphic encryption framework. This allows for the computation of the activation function on encrypted data while preserving data privacy.

Homomorphic Multiplication: For each term x^i in the polynomial, we first need to compute its higher powers using homomorphic multiplication. This is achieved by performing successive homomorphic multiplication operations to calculate $x, x^2, \dots, x^n$ as follows:

$$\text{HomoMult}(\text{EK}, x, x) \rightarrow \text{Enc}(x^2)$$

$$\text{HomoMult}(\text{EK}, \text{Enc}(x^2), x) \rightarrow \text{Enc}(x^3)$$

This process continues iteratively until all required powers are computed. Each multiplication is performed in the encrypted domain, ensuring that the resulting encrypted powers of x can be securely used in subsequent polynomial evaluations.

Homomorphic Multiplication by a Constant: Next, for each x^i , it must be multiplied by its corresponding coefficient a_i . Since the coefficient a_i is in plaintext, this operation can be performed using homomorphic multiplication by a constant:

$$\text{HomoMultConst}(\text{EK}, \text{Enc}(x^i), a_i) \rightarrow \text{Enc}(a_i \cdot x^i)$$

This operation securely computes the product of the encrypted power x^i and the plaintext coefficient a_i , resulting in an encrypted value corresponding to $a_i \cdot x^i$. This step is repeated for each term in the polynomial.

Homomorphic Addition: After calculating all the terms $a_i \cdot x^i$, the next step is to sum these terms using homomorphic addition to obtain the final polynomial value:

$$\sum_{i=0}^n [\text{HomoMultConst}(\text{EK}, \text{Enc}(x_i), a_i)] \rightarrow \text{Result}_{\text{SLAF}}$$

This operation securely adds the encrypted terms together, resulting in the final encrypted output of the SLAF polynomial.

3.4. Construction of homomorphic encryption-friendly privacy-preserving neural network models

In developing efficient, homomorphic encryption-friendly, privacy-preserving neural network models, we have introduced a novel activation function known as SLAF. Concurrently, to enhance the accuracy that is often diminished by the use of square activation function in traditional homomorphic-encryption friendly networks, we have optimized the structure of the linear layers. Our construction process yielded a homomorphic-friendly neural network with 10 layers, designated as SLAFNN-10. This network demonstrates strong feature extraction capabilities and stable training performance, making it well-suited for image classification tasks. To facilitate convolution operations and align the network model with homomorphic encryption objectives, we employed the `im2col` function [36], which restructures input data into matrices. This adjustment reduces the iterations required for convolution operations and modifies the overall network structure, which consists of a modest number of convolutional layers, batch normalization layers, average pooling layers, activation function layers, and fully connected layers.

Considering the linear nature of combined operations within the network, we adopted a hierarchical folding approach similar to the one used in the CryptoNets [5] model. In our constructed model, we eliminate nonlinear operations between the first pooling layer and the first fully connected layer. By folding consecutively deployed linear layers into a single layer, we aim to deepen the network while minimizing computational time. This method results in a condensed version of our original model, SLAFNN-10, now consisting of 5 layers and referred to as SLAFNN-5. The structure of this folded network is detailed in Table 2. To accommodate the operational limitations of homomorphic encryption, which supports only addition and multiplication, we use SLAF as an alternative to traditional activation functions. SLAF, formulated as $a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_nx^n$ with tunable hyperparameter n , incorporates polynomial functions that are easily implementable in homomorphic encryption frameworks. The coefficients $a_0, a_1, a_2, \dots, a_n$ are learned during training to tailor the activation function optimally for various datasets. The implementation of SLAF addresses the common issue of accuracy loss associated with the use of low-order polynomials in traditional homomorphic-encryption networks. The comprehensive design of our homomorphic-encryption friendly, privacy-preserving neural network model is depicted in Fig. 3.

While the strategy of combining successive linear transformation layers significantly lowers the computational demands and resource

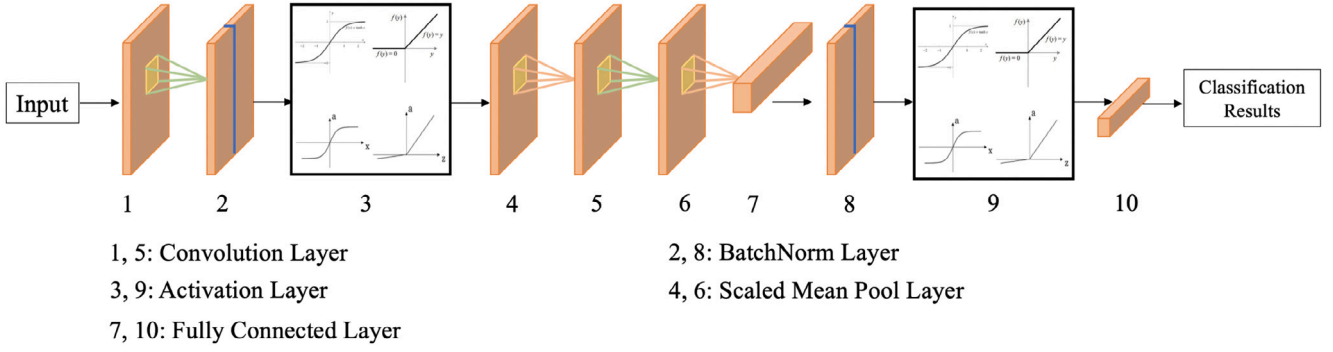


Fig. 3. Homomorphic friendly neural network model for fine-grained access control based on user behavioral features.

Table 2
SLAFNN-10 and SLAFNN-5 neural network model.

	SLAFNN-10	SLAFNN-5
1	Convolution layer	Convolution layer
2	BatchNorm layer	SLAF activation layer
3	SLAF activation layer	
4	Scaled mean pool layer	
5	Convolution layer	Pool layer
6	Scaled mean pool layer	
7	Fully connected layer	
8	BatchNorm layer	
9	SLAF activation layer	SLAF activation layer
10	Fully connected layer	Fully connected layer

usage in neural networks within homomorphic encryption frameworks, the multiple multiplications required by SLAF still demand substantial time. To mitigate this, we developed a simplified neural network model, named SNN, which is optimized for scenarios demanding rapid response times at the cost of slightly reduced accuracy. The SNN model includes one convolutional layer, one square activation layer, and three fully connected layers, making it a streamlined alternative for applications where speed is paramount.

4. Experimental evaluation

The efficacy of our proposed efficient homomorphic encryption-friendly privacy-preserving neural network model has been validated within the domain of biometric identification and prediction. We conducted evaluations using homomorphic encryption on facial image datasets through facial classification experiments. These experiments aimed to simulate the recognition and classification of user behavioral characteristics. We rigorously assessed both the accuracy and latency performance of our classification process. The facial dataset used includes diverse facial occurrences, confirming the practicality and effectiveness of our approach.

MNIST data: The MNIST dataset is a standard image recognition dataset widely used for training and testing various machine learning and deep learning models [40]. It contains 60,000 training images and 10,000 test images, each being a $28 \times 28 \times 1$ pixel image representing handwritten digits from 0 to 9. The MNIST dataset is well-recognized for its standardization and practicality, making it one of the benchmark datasets for evaluating the performance of image classification models. Additionally, the MNIST dataset is used by CryptoNets [5], a neural network model that employs homomorphic encryption for privacy protection. By using the same dataset, we can conduct a fair comparison under identical conditions, further demonstrating the advantages of our model in terms of privacy protection and practicality. This ensures that our experimental results are highly credible and comparable.

UTKFace data: The UTKFace dataset [31] is a large-scale facial dataset spanning a wide age range from 0 to 116 years. Comprising over

20,000 facial images annotated with age, gender, and ethnicity, this dataset encompasses a variety of poses, facial expressions, lighting conditions, occlusions, resolutions, and other factors. It serves as a valuable resource for various tasks including facial detection, age estimation, age progression/regression, and landmark localization. In this paper, we use the UTKFace dataset for classification tasks to simulate fine-grained access control authentication using biometric data in various complex environments.

4.1. Performance indicators

Performance indicators for our model focus on the accuracy and response time critical for effective user behavior pattern recognition classification. The UTKFace dataset was preprocessed into images of dimensions $48 \times 48 \times 1$ and then divided into training and testing sets in an 8:2 ratio, resulting in 18,964 images for training and 4741 for testing. Age was categorized into three groups: 0–15, 16–20, and 21–116 years. We evaluated the accuracy of age, gender, and ethnicity classifications along with computational time costs to approximate the accuracy and response time of user behavior identification classification. Accuracy for face classification was determined by comparing test results from 4741 faces against actual data from the UTKFace dataset, with the final accuracy rate calculated as $NCC \div 4741$, where NCC represents the number of correct classifications.

4.2. SLAF parameter settings

In exploring optimal settings for the SLAF activation function in the SLAFNN model, we considered the computational overhead of homomorphic encryption. We assessed the impact of the highest polynomial degree of the SLAF activation function on both accuracy and computational overhead, utilizing facial datasets for gender, age, and ethnicity classifications. We evaluated four settings corresponding to polynomial degrees of 2, 3, 4, and 5, expressed in general forms from $a_0x^0 + a_1x^1 + a_2x^2$ up to $a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5$, designated as SLAF-2, SLAF-3, SLAF-4, and SLAF-5, respectively. The specific computational overhead for each model is depicted in Fig. 4, while the corresponding accuracy is illustrated in Fig. 5.

Experimental results demonstrate a progressive increase in classification accuracy as the polynomial degree rises from 2 to 4. Notably, a polynomial degree of 4 consistently achieves the highest accuracy across various tasks. However, a decrease in accuracy occurs when the polynomial degree reaches 5, possibly indicating overfitting. As polynomial degrees increase from 2 to 4, there is a continual improvement in model accuracy, likely due to the model's enhanced capability to learn additional parameters. However, the trend reverses at a polynomial degree of 5. In the context of homomorphic encryption, the polynomial degree of 4 not only yields the highest accuracy but also maintains computational overhead comparable to that of degree 3, remaining acceptable relative to degree 2, and significantly lower than degree

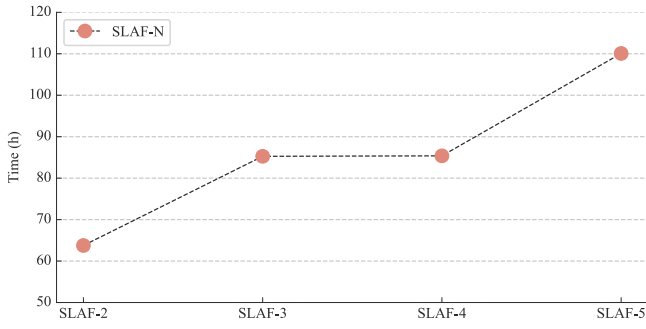


Fig. 4. The comparison of computational overhead among SLAFNN models with different polynomial coefficients for the entire testing set.

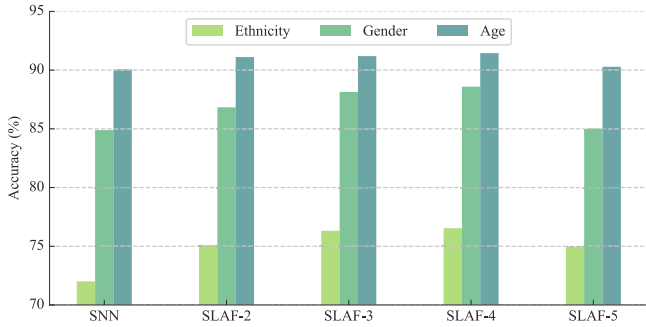


Fig. 5. Comparison of classification accuracy between SNN model and SLAFNN model with different polynomial degrees.

5. Balancing computational cost and accuracy under homomorphic encryption constraints, we opted to set the polynomial degree of the activation function in the SLAFNN model to 4, striking an optimal compromise between time expenditure and accuracy attainment.

4.3. Model network layer settings

With the goal of balancing accuracy and computational overhead, we conducted an in-depth exploration of the network depth configuration. Considering the fusion of linear layers within the network model, we introduced additional convolutional and average pooling layers after the sixth layer of the SLAFNN-10 model. We denoted the models with the introduction of a set of convolutional and pooling layers as SLAFNN-12, and those with the introduction of two sets as SLAFNN-14. The specific computational overhead for networks with different numbers of layers is depicted in Fig. 6, while the corresponding accuracy is illustrated in Fig. 7. The experimental findings indicate that deeper models contribute to enhanced accuracy. However, within the homomorphic encryption framework, the escalation in computational overhead surpasses the improvement in accuracy. Consequently, the SLAFNN-10 model strikes a favorable balance between computational cost and classification accuracy.

4.4. Activation function settings

In order to ascertain the efficacy of the activation function employed in addressing the inherent decrease in accuracy observed in traditional homomorphic-friendly networks when dealing with low-degree polynomial operations, we conducted comparative accuracy tests by maintaining the identical network architecture as SLAFNN-10 while solely substituting the activation function. Models utilizing the square activation function are denoted as SLNN- x^2 , those employing ReLU-AQ [25] are termed SLNN10-AQ, and those utilizing ReLU are referred to as SLNN10-ReLU. The accuracy comparisons among these models

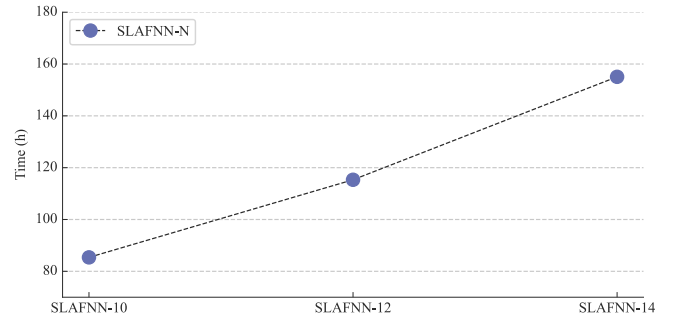


Fig. 6. The comparison of computational overhead among networks with different numbers of layers for the entire testing set.

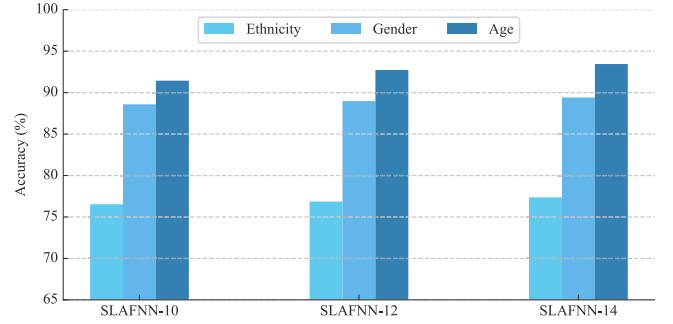


Fig. 7. The comparison of accuracy among networks with different numbers of layers.

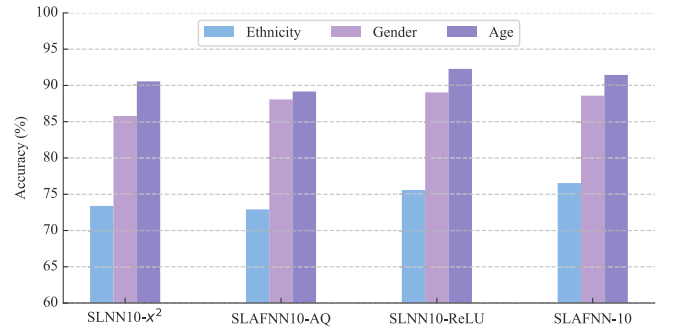


Fig. 8. The comparison of accuracy among networks with different activation function (SLNN10-ReLU is plaintext accuracy).

are depicted in Fig. 8. Experimental findings demonstrate that our proposed network model with the SLAF activation function exhibits an accuracy enhancement ranging from 0.88% to 3.15% compared to models employing the square activation function for encrypted data processing. Furthermore, when compared to ReLU-AQ, accuracy is enhanced by 0.52% to 3.63%. In comparison to models utilizing traditional ReLU functions, our accuracy slightly decreases by 0.45% to 0.84% in gender and age classification tasks; however, it exhibits a performance improvement of 0.96% in ethnicity classification tasks. These experimental results underscore the effectiveness of the SLAF activation function in mitigating the accuracy degradation issue stemming from the utilization of low-degree polynomials in traditional homomorphic-friendly networks.

4.5. Experimental comparisons

In selecting CryptoNets as a benchmark for our experiments, several key considerations influenced our decision. First and foremost, CryptoNets, developed by Microsoft, represents a pioneering neural network model that successfully integrates homomorphic encryption

Table 3

Accuracy of different models on MNIST dataset.

Model	Accuracy on MNIST	Single prediction time
CryptoNets	98.95%	250 s ^a
CryptoNets-ReLU	98.97% ^b	—
CryptoNets-AQ	99.10%	31.23 s
CryptoNets-SLAF	99.09%	32.43
Ours-SNN	97.98%	0.64 s
Ours-SLAFNN	99.12%	34.91 s

^a Represents the accuracy and time overhead when using the YASHE homomorphic encryption scheme.

^b Represents the plaintext test accuracy.

with neural network inference [5]. This integration has established CryptoNets as a crucial benchmark in the domain of privacy-preserving machine learning. The model is widely recognized in both academic and industrial settings due to its high throughput, precision in handling encrypted data, and robust privacy protection capabilities, making it particularly suitable for “Machine Learning as a Service” (MLaaS) applications [41].

Furthermore, the introduction of CryptoNets not only demonstrated the effective combination of homomorphic encryption and neural networks but also spurred subsequent advancements, leading to the development of models such as Faster CryptoNets [25] and CryptoDL [9], which focus on optimizing computational efficiency and operational costs. By comparing our proposed model with CryptoNets, we aim to highlight the improvements we have made in terms of accuracy, thereby underscoring the effectiveness and practicality of our approach.

To evaluate the effectiveness of our proposed approach, we first compared it with the CryptoNets method on the MNIST dataset. Additionally, we applied ReLU-AQ activation functions and Self-Learnable Activation Functions (SLAF) to the CryptoNets neural network to further demonstrate SLAF’s ability to enhance the accuracy of network models. Table 3 shows the accuracy performance of different models on the MNIST dataset.

Although CryptoNets can simultaneously perform 4096 predictions on a PC, achieving a throughput of 58,982 predictions per hour, each individual prediction takes 250 s [5]. Additionally, the YASHE homomorphic encryption scheme used by CryptoNets is no longer considered secure due to attacks proposed by Albrecht et al. [42]. For a fairer comparison, we implemented the CryptoNets neural network using the CKKS homomorphic encryption scheme [22] and tested it on an Apple M1 Pro CPU (8 cores) with 16 GB of memory. The results showed that a single ciphertext prediction using the CryptoNets neural network takes approximately 30.22 s.

In comparison, while the accuracy of the SNN model decreased by 0.97%, this accuracy is still within an acceptable range, and the single prediction time is only 0.64 s, significantly improving response speed. On the other hand, SLAFNN-10 provided the best accuracy performance. Therefore, we recommend using the SNN model in scenarios that require quick responses, while SLAFNN-10 should be applied in scenarios where high accuracy is prioritized. This strategy allows for optimal performance in different application contexts, balancing the needs for response speed and accuracy.

To further demonstrate the practicality and effectiveness of our proposed solution in biometric authentication, we also validated our approach using the UTKFace dataset. We compared our approach with the CryptoNets neural network model and its facial classification variant. In these experiments, we examined the performance of each method in terms of time and accuracy. Table 4 highlights the differences between our method and others from three perspectives: privacy protection, accuracy, and processing speed.

The experimental results indicate that both the SNN model and the CryptoNets model exhibit lower accuracy among the four models due to the constraint imposed by employing only square activation

Table 4

Comparison of functions of different schemes.

Model	Functionality		
	Privacy preserving	Accuracy	Efficiency
CryptoNets	●	○	○
CryptoNets-ReLU	○	●	○
CryptoNets-AQ	●	●	○
Ours-SNN	●	●	●
Ours-SLAFNN	●	●	○

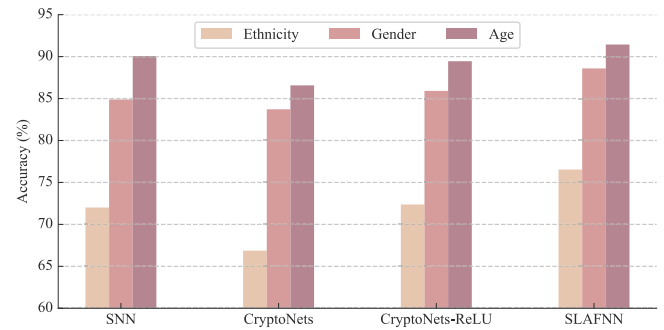


Fig. 9. Comparison of classification accuracy of SNN model, CryptoNets model, CryptoNets-ReLU model and SLAFNN model (CryptoNets-ReLU is plaintext accuracy).

function. We posit that the SNN model achieves higher accuracy compared to the CryptoNets model with fewer network layers primarily because the CryptoNets model, utilizing square activation function twice, encounters the issue of gradient explosion, leading to suboptimal training outcomes. The CryptoNets-ReLU model, which replaces the square activation function in the CryptoNets model with the ReLU activation function, achieves relatively favorable accuracy; however, it still exhibits a certain gap when compared to our proposed SLAFNN model. The specific accuracies of the four models across gender, age, and ethnicity classifications are illustrated in Fig. 9.

Following rigorous experimentation, Table 5 provides a detailed comparison of the different models in terms of accuracy and computational overhead across gender, age, and ethnicity classifications in plaintext and ciphertext states. The experimental findings demonstrate the outstanding accuracy performance of the SLAFNN model. The experimental results demonstrate that our proposed models achieves excellent accuracy while ensuring privacy protection. When performing inference on an Apple M1 Pro CPU (8 cores) with 16 GB memory, the high-precision solution using the SLAFNN model requires approximately 1 min for single-image classification processing. Although a wait time of 1 min is somewhat acceptable, it remains relatively long. Therefore, opting for the rapid-response solution using the SNN model further compromises between accuracy and computational efficiency, allowing for encrypted image processing in approximately 0.778 s. In summary, our proposed solution achieves a commendable balance between accuracy and computational efficiency, meeting the requirements of security, accuracy, and efficiency effectively.

5. Discussion

Multiplication depth: In developing networks utilizing the CKKS scheme—a leveled homomorphic encryption (LHE) scheme—the design requires precise setting of network-level parameters to establish the maximum number of multiplications supported. To achieve optimal efficiency at the 128-bit security level, we calibrated the multiplication depth to its minimal requisite. According to our calculations, the simpler SNN model requires at least 6 multiplication operations, whereas the more complex SLAFNN model needs at least 10. To improve the scalability of these solutions, we also evaluated the efficacy of parameters that support higher security levels and greater multiplication

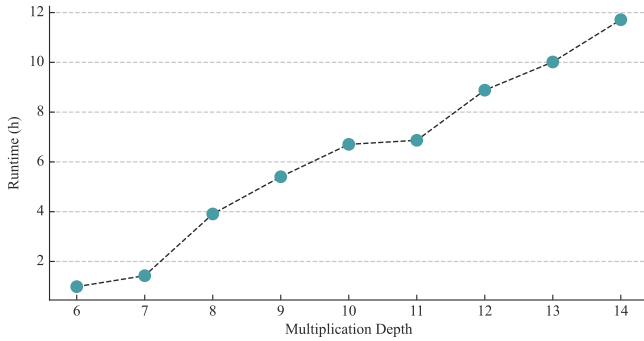
Table 5

Comparison of accuracy and time overhead of different models.

Model	Identify type	Gender		Age		Ethnicity	
		Plaintext	Ciphertext	Plaintext	Ciphertext	Plaintext	Ciphertext
CryptoNets	Accuracy/%	83.78	83.72	86.69	86.57	66.99	66.87
	Time/s	2.2514	245 868.26	2.0561	240 653.2	2.1109	246 792.75
CryptoNets-ReLU	Accuracy/%	85.91	^a	89.45	^a	72.37	^a
	Time/s	2.1979	^a	2.0916	^a	2.0373	^a
SLNN10- x^2	Accuracy/%	85.83	85.78	90.61	90.56	73.51	73.39
	Time/s	0.9457	219 588.89	0.9684	220 382.7	0.9305	220 875.64
SLNN10-AQ	Accuracy/%	88.12	88.07	89.14	89.16	72.96	72.91
	Time/s	0.9467	308 165.24	0.8954	310 438.8	0.9182	304 961.19
SLNN10-ReLU	Accuracy/%	89.04	^a	92.28	^a	75.58	^a
	Time/s	0.781	^a	0.7071	^a	0.8856	^a
Ours-SNN	Accuracy/%	84.86	84.89	90.28	90.07	72.05	72.01
	Time/s	0.1137	3515.73	0.1274	3103.13	0.1226	3553.36
Ours-SLAFNN-10	Accuracy/%	88.65	88.59	91.49	91.44	76.68	76.54
	Time/s	0.7952	307 293.36	0.7582	306 848.9	0.6954	307 377.52

1. The time values presented in the table represent the total testing time across the entire testing set.

2. Due to the inherent difficulty in directly implementing the ReLU function within homomorphic encryption schemes.

^a Denotes the inability to perform ciphertext testing.**Fig. 10.** Time consumption for ciphertext classification with different multiplication depths.

depths within the same network structures. Fig. 10 depicts the time required for making classification predictions in ciphertext state as the multiplication depth varies from 6 to 14 layers within the SNN model setup. These detailed parameter settings and efficiency analyses underpin the theoretical feasibility and performance optimization of our proposed models.

Accuracy: Homomorphic encryption provides robust security and computational capabilities, but it also entails significant computational demands, especially for Convolutional Neural Network (CNN) computations involving high-resolution images. To address this, we proposed two models: the SNN model and the SLAFNN model. When tested on an Apple M1 Pro CPU (8 cores) with 16 GB of memory, the SLAFNN model demonstrated excellent accuracy, but each processing time was approximately one minute during biometric authentication simulations. In contrast, the SNN model could process a single encrypted image in just 0.778 s.

To demonstrate the practicality of the SNN model in scenarios requiring rapid response, we compared the SNN model with the CryptoNets model on the MNIST dataset. The results showed that the SNN model's accuracy was only 0.97% lower than the CryptoNets model, but its speed was significantly improved. Furthermore, the SNN model exhibited higher accuracy than the CryptoNets model on the UTKFace dataset, indicating its adaptability and accuracy across different scenarios. The SLAFNN model achieved the highest accuracy on both the MNIST and UTKFace datasets, while also performing well in terms of computation time.

Overall, the SNN model is suitable for scenarios requiring quick responses, while the SLAFNN model is more appropriate for scenarios

demanding higher accuracy. The experimental results further validate the practicality and effectiveness of our proposed solutions, showcasing their advantages in privacy protection and efficient computation.

Extensions: Looking forward, although current major homomorphic cryptographic libraries are optimized for CPU usage, our research also plans to expand into GPU implementations to further decrease runtime. For instance, the work by Wonkyung Jung et al. [43] demonstrated a significant speedup by implementing CKKS bootstrapping on a GPU, achieving a 40.0× faster processing time compared to an 8-threaded CPU setup. This future work aims to enhance the computational efficiency of our schemes and expand their applicability in homomorphic cryptography across a broader range of applications. These advancements are anticipated to provide substantial performance improvements and expand the practical utility of homomorphic encryption technologies.

6. Conclusion

In this study, we have introduced an integration of Convolutional Neural Networks (CNN) with Fully Homomorphic Encryption (FHE) to develop a robust privacy-preserving mechanism. We designed efficient neural network models that incorporate “SLAF” as the activation function within the homomorphic encryption framework, blending the strengths of both CNN and FHE. Through comprehensive experimentation, we assessed the impact of various parameters and the depth of the SLAF model on both accuracy and computational overhead. This analysis helped us to identify the optimal balance between accuracy and response time.

Our evaluations, conducted using the UTKFace facial database [31], robustly demonstrated the effectiveness of our proposed approach. To accommodate varying requirements across different scenarios, we developed two privacy-preserving network models. The first model, leveraging the SLAFNN model, provides excellent accuracy with acceptable waiting times. The second, employing the SNN model, features response times as rapid as 0.778 s. Although this model shows some performance degradation, it remains a competitive option within the field of privacy-preserving computational models.

CRedit authorship contribution statement

Jichao Xiong: Writing – original draft, Visualization, Validation, Methodology, Investigation. **Jiageng Chen:** Writing – review & editing, Validation, Supervision, Methodology. **Junyu Lin:** Validation, Resources, Project administration. **Dian Jiao:** Software, Formal analysis, Data curation, Conceptualization. **Hui Liu:** Validation, Supervision, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is financially supported by self-determined research funds of CCNU from the colleges' basic research and operation of MOE CCNU24ai010, CCNU22JC001, and the National Natural Science Foundation of China under Grant No. 62272189.

References

- [1] Morampudi MK, Prasad MV, Verma M, Raju U. Secure and verifiable iris authentication system using fully homomorphic encryption. *Comput Electr Eng* 2021;89:106924.
- [2] Choi H, Woo SS, Kim H. Blind-touch: Homomorphic encryption-based distributed neural network inference for privacy-preserving fingerprint authentication. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38, 2024, p. 21976–85.
- [3] Pradel G, Mitchell C. Privacy-preserving biometric matching using homomorphic encryption. In: *2021 IEEE 20th international conference on trust, security and privacy in computing and communications*. trustCom, IEEE; 2021, p. 494–505.
- [4] Nguyen DTK, Duong DH, Susilo W, Chow Y-W, Ta TA. HeFUN: Homomorphic encryption for unconstrained secure neural network inference. *Future Internet* 2023;15(12):407.
- [5] Gilad-Bachrach R, Dowlin N, Laine K, Lauter K, Naehrig M, Wernsing J. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In: *International conference on machine learning*. PMLR; 2016, p. 201–10.
- [6] Deng L. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Process Mag* 2012;29(6):141–2.
- [7] Lee J-W, Kang H, Lee Y, Eom J, Deryabin M, Lee E, Lee J, Yoo D, Kim Y-S, et al. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* 2022;10:30039–54.
- [8] van Elsloo T, Patrini G, Ivey-Law H. SEALion: A framework for neural network inference on encrypted data. 2019, arXiv preprint arXiv:1904.12840.
- [9] Hesamifard E, Takabi H, Ghasemi M. Cryptodl: Deep neural networks over encrypted data. 2017, arXiv preprint arXiv:1711.05189.
- [10] Knott B, Venkataraman S, Hannun A, Sengupta S, Ibrahim M, van der Maaten L. Crypten: Secure multi-party computation meets machine learning. *Adv Neural Inf Process Syst* 2021;34:4961–73.
- [11] Xu R, Baracaldo N, Zhou Y, Anwar A, Ludwig H. Hybridalpha: An efficient approach for privacy-preserving federated learning. In: *Proceedings of the 12th ACM workshop on artificial intelligence and security*. 2019, p. 13–23.
- [12] Choi H, Woo SS, Kim H. Blind-touch: Homomorphic encryption-based distributed neural network inference for privacy-preserving fingerprint authentication. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38, 2024, p. 21976–85.
- [13] Ran R, Xu N, Liu T, Wang W, Quan G, Wen W. Penguin: Parallel-packed homomorphic encryption for fast graph convolutional network inference. *Adv Neural Inf Process Syst* 2024;36.
- [14] Niu Y, Deng W. Federated learning for face recognition with gradient correction. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 36, 2022, p. 1999–2007.
- [15] Tran A-T, Luong T-D, Karnjana J, Huynh V-N. An efficient approach for privacy preserving decentralized deep learning models based on secure multi-party computation. *Neurocomputing* 2021;422:245–62.
- [16] Tang X, Zhu L, Shen M, Peng J, Kang J, Niyato D, Abd El-Latif AA. Secure and trusted collaborative learning based on blockchain for artificial intelligence of things. *IEEE Wirel Commun* 2022;29(3):14–22.
- [17] Kumar R, Kumar J, Khan AA, Ali H, Bernard CM, Khan RU, Zeng S, et al. Blockchain and homomorphic encryption based privacy-preserving model aggregation for medical images. *Comput Med Imaging Graph* 2022;102:102139.
- [18] Ku H, Susilo W, Zhang Y, Liu W, Zhang M. Privacy-preserving federated learning in medical diagnosis with homomorphic re-encryption. *Comput Stand Interfaces* 2022;80:103583.
- [19] Brakerski Z, Gentry C, Vaikuntanathan V. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory (TOCT)* 2014;6(3):1–36.
- [20] Fan J, Vercauteren F. Somewhat practical fully homomorphic encryption. *Cryptol ePrint Arch* 2012.
- [21] Chillotti I, Gama N, Georgieva M, Izabachène M. TFHE: fast fully homomorphic encryption over the torus. *J Cryptology* 2020;33(1):34–91.
- [22] Cheon JH, Kim A, Kim M, Song Y. Homomorphic encryption for arithmetic of approximate numbers. In: *Advances in cryptography—ASIACRYPT 2017: 23rd international conference on the theory and applications of cryptology and information security*, Hong Kong, China, December 3–7, 2017, proceedings, part I 23. Springer; 2017, p. 409–37.
- [23] Al Badawi A, Jin C, Lin J, Mun CF, Jie SJ, Tan BHM, Nan X, Aung KMM, Chandrasekhar VR. Towards the alexnet moment for homomorphic encryption: Hcnn, the first homomorphic cnn on encrypted data with gpus. *IEEE Trans Emerg Top Comput* 2020;9(3):1330–43.
- [24] Xie P, Wu B, Sun G. BAYHENN: Combining Bayesian deep learning and homomorphic encryption for secure DNN inference. 2019, arXiv preprint arXiv:1906.00639.
- [25] Chou E, Beal J, Levy D, Yeung S, Haque A, Fei-Fei L. Faster cryptonets: Leveraging sparsity for real-world encrypted inference. 2018, arXiv preprint arXiv:1811.09953.
- [26] Lee J, Lee E, Lee J-W, Kim Y, Kim Y-S, No J-S. Precise approximation of convolutional neural networks for homomorphically encrypted data. *IEEE Access* 2023.
- [27] Wang C-C, Tu C-H, Kao M-C, Hung S-H. TensorHE: a homomorphic encryption transformer for privacy-preserving deep learning. In: *Proceedings of the conference on research in adaptive and convergent systems*. 2022, p. 124–30.
- [28] Dathathri R, Saarikivi O, Chen H, Laine K, Lauter K, Maleki S, Musuvathi M, Mytkowicz T. CHET: an optimizing compiler for fully-homomorphic neural-network inferencing. In: *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*. 2019, p. 142–56.
- [29] Lu W-j, Huang Z, Hong C, Ma Y, Qu H. PEGASUS: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In: *2021 IEEE symposium on security and privacy*. SP, IEEE; 2021, p. 1057–73.
- [30] Boura C, Gama N, Georgieva M, Jetchev D. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *J Math Cryptol* 2020;14(1):316–38.
- [31] Zhang Z, Song Y, Qi H. Age progression/regression by conditional adversarial autoencoder. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, p. 5810–8.
- [32] Rivest RL, Adleman L, Dertouzos ML, et al. On data banks and privacy homomorphisms. *Found Secur Comput* 1978;4(11):169–80.
- [33] Gentry C. A fully homomorphic encryption scheme. Stanford University; 2009.
- [34] Benaissa A, Retiat B, Cebere B, Belfedhal AE. TenSEAL: A library for encrypted tensor operations using homomorphic encryption. 2021, arXiv:2104.03152.
- [35] SEAL. Microsoft SEAL (release 3.7). 2021, <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA.
- [36] Forn T, Fontova-Musté P, Caro M, Abella J, Moll F, Altet J, Studer C. An energy-efficient GeMM-based convolution accelerator with on-the-fly im2col. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 2023;31(11):1874–8. <http://dx.doi.org/10.1109/TVLSI.2023.3286122>.
- [37] Halevi S, Shoup V. Algorithms in helib. In: *Advances in cryptography—CRYPTO 2014: 34th annual cryptography conference*, Santa Barbara, CA, USA, August 17–21, 2014, proceedings, part I 34. Springer; 2014, p. 554–71.
- [38] Goyal M, Goyal R, Lall B. Learning activation functions: A new paradigm of understanding neural networks. 2019, arXiv abs/1906.09529. URL: <https://api.semanticscholar.org/CorpusID:195345613>.
- [39] Goyal M, Goyal R, Lall B. Improved polynomial neural networks with normalised activations. In: *2020 international joint conference on neural networks. IJCNN, IEEE*; 2020, p. 1–8.
- [40] Deng L. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Process Mag* 2012;29(6):141–2.
- [41] Tanuwidjaja HC, Choi R, Kim K. A survey on deep learning techniques for privacy-preserving. In: *Machine learning for cyber security: second international conference, ML4CS 2019, Xi'an, China, September 19–21, 2019, proceedings 2*. Springer; 2019, p. 29–46.
- [42] Albrecht M, Bai S, Ducas L. A subfield lattice attack on overstretched NTRU assumptions: Cryptanalysis of some FHE and graded encoding schemes. In: *Annual international cryptography conference*. Springer; 2016, p. 153–78.
- [43] Jung W, Kim S, Ahn JH, Cheon JH, Lee Y. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus. In: *IACR transactions on cryptographic hardware and embedded systems*. 2021, p. 114–48.